Supplemental Material for "Recovering a time-dependent control field from qubit expectation values: physics-informed neural networks versus classical inversion"

Gubio G. de Lima[1]

[1] *Physics Department, Federal University of São Carlos, 13565-905, São Carlos, SP, Brazil*

(Dated: November 6, 2026)

This document gives the technical details and all numerical values needed to reproduce the results of the main text. Every number below is produced by the scripts of the code repository (https://github.com/GubioGL/Pinn-inverse-for-H-time): `run/rodar.py` runs the benchmark, `run/tabelas_artigo.py` writes the tables and `run/fig_artigo.py` the figures. Source files are indicated as `src/<file>.py`.

CONTENTS

S1. Data generation and units 2

S2. Identifiability 3

S3. Field representations 4

S4. Physics-informed neural network 6

S5. Optimizers 7

S6. Classical estimators 9

S7. Protocol and reproducibility 10

S8. Additional results 10

S9. Outlook settings: details 16

References 16

S1. DATA GENERATION AND UNITS

a. Model and units. The Hamiltonian is $H(t) = \frac{1}{2}\sigma_z + \frac{1}{2}f(t)\sigma_x$ with $\hbar = 1$ and unit qubit splitting; time is measured in units of the inverse splitting and the field f in units of the splitting. For a pure state, the Bloch vector $\boldsymbol{r} = (X, Y, Z)$ obeys

$$\dot{\boldsymbol{r}} = \boldsymbol{\Omega}(t) \times \boldsymbol{r}, \qquad \boldsymbol{\Omega} = (f, 0, 1), \tag{S1}$$

which is equivalent to the Schrödinger (or von Neumann) equation for this Hamiltonian.

b. Test fields. Table S1 lists the fields. Field 3 and the single-tone pulses of the spectral study (`"w<omega>"` cases in `src/bloch.py`) share the envelope $\sin^2(\pi t/T)$, so $f(0) = f(T) = 0$; this is a property of the pulse design and is the only information about the field used by the edge constraint of Sec. S3 0 g.

TABLE S1. Test fields. All are evaluated on $N = 100$ equally spaced times $t_i \in [0, T]$.

field	$f(t)$	T	remarks
1	$\cos t$	2π	periodic on $[0, T]$, single harmonic
2	$1.5\sin 2t + 0.8\cos(3.5t + 0.5) + 0.4\sin 7t$	2.52π	$f(0) \neq f(T)$, not periodic
3	$[1.5\sin 2t - 0.3\cos(3t - 0.5) + 0.4\sin 7t]\,\sin^2(\pi t/T)$	2.52π	smooth pulse, $f(0) = f(T) = 0$
ω	$\sin(\omega t)\,\sin^2(\pi t/T)$	2.52π	spectral study, $\omega \in \{1, \ldots, 20\}$

c. Integration. Equation (S1) is integrated with the eighth-order Dormand–Prince method (SciPy `solve_ivp`, method `DOP853`) with relative tolerance 10^{-11} and absolute tolerance 10^{-12} (`src/bloch.py::simulate`). As a cross-check, the same trajectories obtained with QuTiP `mesolve` [1] differ by less than 5×10^{-11} when the field is passed as a function, and by 8×10^{-6} when it is passed as an array sampled at the data times (QuTiP then interpolates it with cubic splines).

d. Noise. Noisy data are $\boldsymbol{r}_i = \boldsymbol{r}(t_i) + \boldsymbol{\eta}_i$ with independent Gaussian components of standard deviation $\sigma \in \{0, 0.01, 0.03\}$. For reference, $\sigma = 0.01$ corresponds to the projection noise of $\sim 10^4$ single-shot measurements per expectation value.

e. Seeds. For each (field, σ, seed s) the data are generated once, with the random generator seeded by $10^5 c + 10^3 \lfloor 10^3 \sigma \rceil + s$, where c is the field index (`src/executar.py::gerar_dados`). All methods receive the same data. The seed $s \in \{0, \ldots, 4\}$ also sets the initialization of every network and the random collocation points.

S2. IDENTIFIABILITY

a. Pointwise inversion and sensitivity. The second and third components of Eq. (S1) read $fZ = X - \dot{Y}$ and $fY = \dot{Z}$. Minimizing $(fZ - X + \dot{Y})^2 + (fY - \dot{Z})^2$ over f at each instant gives

$$f = \frac{Z(X - \dot{Y}) + Y\dot{Z}}{Y^2 + Z^2}. \tag{S2}$$

The derivative of the right-hand side of Eq. (S1) with respect to f is $(0, -Z, Y)$, whose squared norm $Y^2 + Z^2 = 1 - X^2$ measures the local sensitivity of the dynamics to the field. It vanishes when the Bloch vector is parallel to the control axis. For the initial state $|+\rangle$ it vanishes at $t = 0$; Table S2 quantifies this for the three test fields.

TABLE S2. Fraction of the interval in which $Y^2 + Z^2 < 0.05$, and minimum of $Y^2 + Z^2$, for three initial states (computed on 400 equally spaced times; `run/diagnostico.py`).

	$\|+\rangle$		$\|0\rangle$		$(1,1,1)/\sqrt{3}$	
field	fraction	minimum	fraction	minimum	fraction	minimum
1	9.0%	0	0%	0.38	6.0%	0.018
2	7.0%	0	0%	0.13	0%	0.26
3	4.0%	0	0%	0.52	5.2%	0.011

b. Initial state and measurement budget. Table S3 compares designs with the same total number of measurements: splitting the budget over more time points or more initial states increases the noise per point by $\sqrt{(\text{number of points})/100}$. Preparing $|0\rangle$ instead of $|+\rangle$ halves the error of linear least squares and reduces its spread over noise realizations by a factor of six; three initial states perform like $|0\rangle$ with LSQ and better with shooting at the higher noise level.

c. Symmetries. Two symmetries of Eq. (S1) map a solution for f onto a solution for $-f$:

$$(X, Y, Z, f) \to (-X, -Y, Z, -f), \qquad \text{(rotation by } \pi \text{ about } z;\ \sigma_z H(f)\sigma_z = H(-f)), \tag{S3}$$

$$(X, Y, Z, f) \to (X, Y, -Z, -f). \tag{S4}$$

The first leaves invariant the initial states with $X = Y = 0$ ($|0\rangle$, $|1\rangle$) and preserves $Z(t)$; the second leaves invariant the initial states with $Z = 0$ (e.g. $|+\rangle$) and preserves $X(t)$ and $Y(t)$. Consequently, the sign of f cannot be recovered from $\langle\sigma_z\rangle$ alone when starting from $|0\rangle$, nor from $\langle\sigma_x\rangle$ and $\langle\sigma_y\rangle$ when starting from $|+\rangle$. Numerically, for the three test fields the invariant components agree to machine precision, while the others differ by 0.9–2; for the generic state $(1,1,1)/\sqrt{3}$ all components differ by 0.8–1.5.

TABLE S3. Field 3: relative field error (mean ± standard deviation over five noise realizations) for designs with the same measurement budget; σ_{ref} is the noise of the reference design (one state, $N = 100$). Linear least squares with 40 cubic B-splines and shooting initialized by it (`run/parte1_classicos.py`).

design	σ per point	$\sigma_{\mathrm{ref}} = 0.01$		$\sigma_{\mathrm{ref}} = 0.03$	
		LSQ	shooting	LSQ	shooting
$\lvert+\rangle$, $N = 100$	σ	0.153 ± 0.053	0.093 ± 0.010	0.311 ± 0.151	0.225 ± 0.018
$\lvert 0\rangle$, $N = 100$	σ	0.069 ± 0.008	0.067 ± 0.005	0.151 ± 0.024	0.194 ± 0.012
$\lvert+\rangle$, $N = 300$	$\sqrt{3}\,\sigma$	0.190 ± 0.062	0.073 ± 0.009	0.245 ± 0.087	0.198 ± 0.027
$\lvert+\rangle, \lvert 0\rangle, \lvert+i\rangle$, $N = 100$ each	$\sqrt{3}\,\sigma$	0.073 ± 0.012	0.069 ± 0.015	0.158 ± 0.021	0.151 ± 0.031

d. Sensitivity to the frequency of the field. Figure S1 shows the RMS change of the data produced by a perturbation $a \sin(\omega t) \sin^2(\pi t/T)$ of field 3, per unit a, as a function of ω (finite differences with $a = 10^{-4}$). The decay is close to $1/\omega$: the forward map integrates the field, so high frequencies are weakly encoded in the data.

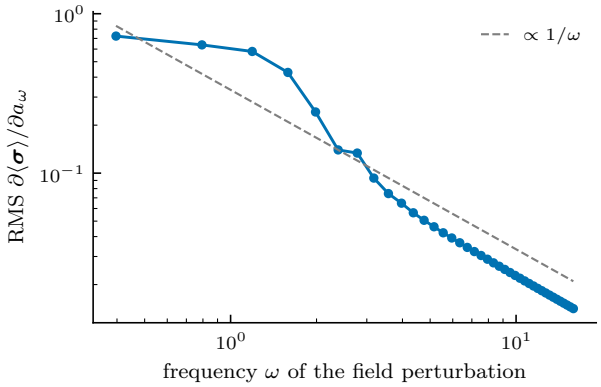


FIG. S1. Sensitivity of the data to a perturbation of frequency ω of field 3 (initial state $|0\rangle$).

S3. FIELD REPRESENTATIONS

a. Fourier series ($M = 10$, 21 coefficients):

$$f(t) = a_0 + \sum_{m=1}^{M} \left[a_m \cos(2\pi m t/T) + b_m \sin(2\pi m t/T) \right], \tag{S5}$$

with highest angular frequency $2\pi M/T = 7.9$.

b. Cosine series ($M = 40$, 41 coefficients): $f(t) = \sum_{m=0}^{M} c_m \cos(m\pi t/T)$, the even extension of f to period $2T$. Highest angular frequency $\pi M/T = 15.9$.

c. Cubic B-splines ($n = 80$, $k = 3$): $f(t) = \sum_{j=1}^{n} c_j B_{j,3}(t)$ on the open uniform knot vector with $k + 1 = 4$ repeated knots at each end and $n - k - 1 = 76$ equally spaced interior knots (spacing $h = T/77 = 0.103$ for $T = 2.52\pi$). The basis is evaluated by the Cox–de Boor recursion [2], implemented in PyTorch (`src/pinn.py::bspline_torch`); it agrees with SciPy's `BSpline.design_matrix` to 2×10^{-10}. The original notebooks used degree $k = 10$ with $n = 30$, which is less accurate for the same number of coefficients (Fig. S2).

d. Neural-network field. An extra (fourth) output of the state network (Sec. S4); 33 additional parameters.

e. MixFunn. Two MixFunn layers [3] ($1 \to 4 \to 1$ neurons) with the function dictionary $\{\sin x, \cos x, e^{-0.01|x|}, x\}$ and second-order neurons (products of pairs of dictionary functions, $4 \cdot 5/2 = 10$ products), i.e. $4 + 10 = 14$ mixing weights per neuron; 274 parameters in total. The input is the raw time t in the Adam-only recipe (as in the original code) and the normalized time s in the improved recipes. Mixing weights are initialized from a standard normal distribution and the linear projections with PyTorch's default initialization.

f. Capacity of the bases. Figure S2 shows the best L_2 approximation error of single-tone pulses $\sin(\omega t)\sin^2(\pi t/T)$ by each basis. The Fourier basis with $M = 10$ cannot represent frequencies above 7.9; cubic B-splines with $n = 80$ functions represent the pulses with error below 10^{-2} up to $\omega \approx 15$. For the test fields, the best-approximation errors of field 2 are 0.14 (Fourier, $M = 10$), 1.1×10^{-2} (cosine, $M = 40$) and 6.9×10^{-5} (B-spline, $n = 80$); for field 3 they are 4.9×10^{-3}, 7.0×10^{-5} and 8.5×10^{-5}, respectively.

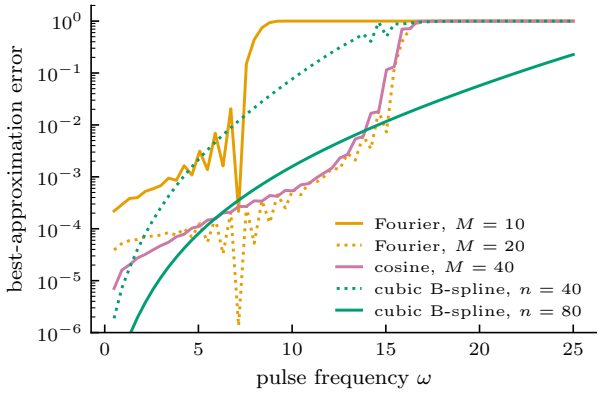


FIG. S2. Best-approximation error of single-tone pulses by each field basis.

g. Edge constraint. When $a = f(0)$ and $b = f(T)$ are known, every representation g is replaced by

$$f(t) = g(t) - (1 - \tau)\, g(0) - \tau\, g(T) + (1 - \tau)\, a + \tau\, b, \qquad \tau = t/T, \tag{S6}$$

which satisfies $f(0) = a$ and $f(T) = b$ exactly and is linear in g. For the bases, $f = \tilde{\mathsf{B}}\boldsymbol{c} + \boldsymbol{m}$ with $\tilde{B}_j(t) = B_j(t) - (1-\tau)B_j(0) - \tau B_j(T)$ and $m(t) = (1-\tau)a + \tau b$. The constraint is applied only to field 3 and to the spectral pulses ($a = b = 0$).

TABLE S4. Number of trainable parameters: field only, and total (state network plus field) for the two network architectures (Table S5).

representation	field	total, Adam-only recipe	total, improved recipes
NN field	16 / 33	334	2308
Fourier ($M = 10$)	21	339	2296
cosine ($M = 40$)	41	359	2316
B-spline ($n = 80$)	80	398	2355
MixFunn	274	592	2549

S4. PHYSICS-INFORMED NEURAL NETWORK

a. State network. A fully connected network with L hidden layers of width W and sine activations maps the input to three outputs (four when the field is a network output). Weights and biases of each layer are drawn from $\mathcal{U}(-1/\sqrt{n_{\text{in}}}, 1/\sqrt{n_{\text{in}}})$, where n_{in} is the input width of the layer (PyTorch's default). Improved recipes: $L = 3$, $W = 32$, input $s = 2t/T - 1$, double precision. Adam-only recipe: $L = 2$, $W = 15$, input t, single precision.

b. Initial condition. Improved recipes impose $\boldsymbol{r}(0) = \boldsymbol{r}_0$ exactly,

$$\boldsymbol{r}_{\boldsymbol{\theta}}(t) = \boldsymbol{r}_0 + \frac{t}{T}\mathcal{N}_{\boldsymbol{\theta}}(s), \qquad \dot{\boldsymbol{r}}_{\boldsymbol{\theta}}(t) = \frac{1}{T}\mathcal{N}_{\boldsymbol{\theta}}(s) + \frac{t}{T}\frac{2}{T}\mathcal{N}'_{\boldsymbol{\theta}}(s). \tag{S7}$$

The Adam-only recipe has no such constraint; the initial condition enters only through the data point at $t = 0$.

c. Time derivative. For a layer $\boldsymbol{h}_{l+1} = \sin(\mathsf{W}_l\boldsymbol{h}_l + \boldsymbol{b}_l)$, the time derivative is propagated together with the activations, $\dot{\boldsymbol{h}}_{l+1} = \cos(\mathsf{W}_l\boldsymbol{h}_l + \boldsymbol{b}_l) \odot \mathsf{W}_l\dot{\boldsymbol{h}}_l$, starting from $\dot{\boldsymbol{h}}_0 = ds/dt = 2/T$. This forward-mode derivative is exact (it agrees with reverse-mode automatic differentiation to 10^{-17}) and requires a single pass, whereas the usual approach differentiates the network output with respect to its input by a backward pass that must itself be differentiated during training.

d. Loss and residual vector. With collocation set $\mathcal{C}$, the loss is

$$\mathcal{L} = \frac{1}{|\mathcal{C}|}\sum_{t\in\mathcal{C}} \|\boldsymbol{R}(t)\|^2 + \frac{w}{N}\sum_{i=1}^{N} \|\boldsymbol{r}_{\boldsymbol{\theta}}(t_i) - \boldsymbol{r}_i\|^2 + \frac{\lambda_f h_r}{T}\sum_{k=2}^{n_r-1}\Big(\frac{f_{k+1} - 2f_k + f_{k-1}}{h_r^2}\Big)^2, \tag{S8}$$

where $\boldsymbol{R} = \dot{\boldsymbol{r}}_{\boldsymbol{\theta}} - \boldsymbol{\Omega}_{\boldsymbol{\theta}} \times \boldsymbol{r}_{\boldsymbol{\theta}}$, $w = 1$, and f_k is the field on a fixed uniform grid of $n_r = 200$ points with spacing h_r; the last term approximates $(\lambda_f/T)\int f''^2 dt$. Equation (S8) is exactly $\|\boldsymbol{\rho}\|^2$ for the residual vector $\boldsymbol{\rho}$ obtained by stacking $\boldsymbol{R}(t)/\sqrt{|\mathcal{C}|}$, $\sqrt{w/N}(\boldsymbol{r}_{\boldsymbol{\theta}}(t_i) - \boldsymbol{r}_i)$ and the scaled second differences. Its length is $3|\mathcal{C}|+3N$ ($+\,n_r-2$ with noise); with $|\mathcal{C}| = 300$ and $N = 100$, $\dim \boldsymbol{\rho} = 1200$ without noise.

e. Collocation. $\mathcal{C}$ contains the N data times and $n_{\text{rand}} = 200$ times drawn uniformly in $[0, T]$, redrawn at every Adam epoch (3000 draws, i.e. 6×10^5 distinct points). L-BFGS, BFGS and LM require a fixed objective during the line search or the trust-region test and use the last draw; LM can optionally redraw every K iterations. The Adam-only recipe uses the data times only.

f. Smoothness weight. λ_f is fixed a priori by the noise level: $\lambda_f = 0$ ($\sigma = 0$), 10^{-6} ($\sigma = 0.01$) and 10^{-5} ($\sigma = 0.03$). These values correspond to the coefficient penalty of the classical LSQ estimator (Sec. S6), $\lambda_f \approx \lambda_D h^3 T/N$ for knot spacing h.

g. Least-squares initialization and variable projection. For a linear representation $f = \tilde{\mathsf{B}}\boldsymbol{c} + \boldsymbol{m}$ and given $(X, Y, Z, \dot{Y}, \dot{Z})$ at n points, the part of Eq. (S8) that depends on $\boldsymbol{c}$ is quadratic, and its minimizer solves

$$\left[\tfrac{1}{n}\tilde{\mathsf{B}}^\top \mathrm{diag}(Y^2+Z^2)\tilde{\mathsf{B}} + s^2(\mathsf{D}_2\tilde{\mathsf{B}}_r)^\top \mathsf{D}_2\tilde{\mathsf{B}}_r\right]\boldsymbol{c} = -\tfrac{1}{n}\tilde{\mathsf{B}}^\top\left[Z(\dot{Y}-X+mZ) - Y(\dot{Z}-mY)\right] - s^2(\mathsf{D}_2\tilde{\mathsf{B}}_r)^\top \mathsf{D}_2 \boldsymbol{m}_r, \tag{S9}$$

with $s^2 = \lambda_f h_r/T$, D_2 the second-difference matrix divided by h_r^2, and $\tilde{\mathsf{B}}_r$, $\boldsymbol{m}_r$ the basis and offset on the regularization grid. The *initialization* uses the data and their cubic-spline derivatives (smoothing spline with generalized cross-validation for noisy data); *variable projection* [4] uses the current network state at the collocation points at every Adam epoch.

h. Optional terms. Causal weights [5]: the collocation points are sorted in time and split into 20 blocks with mean residuals $\mathcal{L}_k$; block k is weighted by $\exp(-\epsilon \sum_{j<k} \mathcal{L}_j)$ with $\epsilon = 1$ (weights not differentiated). Gradient-norm balancing [6]: every 100 Adam epochs, $w \leftarrow (1-\alpha)w + \alpha \max|\nabla \mathcal{L}_{\text{ODE}}|/\overline{|\nabla \mathcal{L}_{\text{data}}|}$ with $\alpha = 0.1$; w is reset to 1 after Adam so that the second stage minimizes the same objective for all methods.

S5. OPTIMIZERS

All optimizers act on a single flat parameter vector (network and field). Table S5 summarizes the recipes; details follow.

a. Adam [7] with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. Improved recipes: 3000 epochs, learning rate decaying from 10^{-3} to 10^{-5} with a cosine schedule. Adam-only recipe: 20000 epochs at a

TABLE S5. Training recipes (generated from `src/protocolo.py`).

item	Adam-only recipe (as in common practice)	improved recipes
state network	2 hidden layers × 15, sin	3 hidden layers × 32, sin
network input	t	$s = 2t/T - 1$
initial condition	soft (data term)	hard, $\boldsymbol{r}(t) = \boldsymbol{r}_0 + (t/T)\,\mathrm{NN}$
field edges	free	hard when known ($f(0) = f(T) = 0$ for field 3)
collocation points	the $N = 100$ data points	N data points + 200 uniform random points, redrawn every Adam epoch
field coefficients at start	zero	linear least squares on the data
floating point	single (float32)	double (float64)
Adam	20000 epochs, learning rate 1×10^{-3}, constant	3000 epochs, learning rate $1 \times 10^{-3} \to 1 \times 10^{-5}$ (cosine schedule)
second stage	none	L-BFGS 5000 it. (history 50, strong Wolfe); BFGS 8000 it.; LM 600 it.
smoothness weight λ_f	$\sigma = 0$: 0; $\sigma = 0.01$: 1×10^{-6}; $\sigma = 0.03$: 1×10^{-5}	
time limit per run	420 s (single CPU thread)	

constant learning rate 10^{-3}.

b. L-BFGS [8] (PyTorch implementation): unit initial step, history 50, strong-Wolfe line search, at most 5000 iterations and 6250 function evaluations, gradient tolerance 10^{-15}, change tolerance 10^{-18}.

c. Dense BFGS. The inverse Hessian H is stored as a dense matrix. After the first step, H is reset to $(\boldsymbol{s}^\top\boldsymbol{y}/\boldsymbol{y}^\top\boldsymbol{y})\mathsf{I}$; every iteration uses a strong-Wolfe line search (at most 30 evaluations) and the update

$$\mathsf{H} \leftarrow \tau\Big[\mathsf{H} - \frac{\mathsf{H}\boldsymbol{y}\boldsymbol{y}^\top\mathsf{H}}{\boldsymbol{y}^\top\mathsf{H}\boldsymbol{y}} + (\boldsymbol{y}^\top\mathsf{H}\boldsymbol{y})\,\boldsymbol{v}\boldsymbol{v}^\top\Big] + \frac{\boldsymbol{s}\boldsymbol{s}^\top}{\boldsymbol{s}^\top\boldsymbol{y}}, \qquad \boldsymbol{v} = \frac{\boldsymbol{s}}{\boldsymbol{s}^\top\boldsymbol{y}} - \frac{\mathsf{H}\boldsymbol{y}}{\boldsymbol{y}^\top\mathsf{H}\boldsymbol{y}}, \tag{S10}$$

with $\tau = 1$ (BFGS) or $\tau = \min(1, \boldsymbol{s}^\top\boldsymbol{y}/\boldsymbol{y}^\top\mathsf{H}\boldsymbol{y})$ (self-scaled variant, in the spirit of Refs. [9, 10]; it is not the exact SSBroyden update of those works). H is reset after a failed line search, and the run stops after three consecutive failures or 8000 iterations.

d. Levenberg–Marquardt [11, 12]. With $\mathsf{J} = \partial\boldsymbol{\rho}/\partial\boldsymbol{\theta}$ ($R \times P$), the step is $\delta\boldsymbol{\theta} = -(\mathsf{J}^\top\mathsf{J} + \mu\mathsf{I})^{-1}\mathsf{J}^\top\boldsymbol{\rho}$ when $P \le R$ and $\delta\boldsymbol{\theta} = -\mathsf{J}^\top(\mathsf{J}\mathsf{J}^\top + \mu\mathsf{I})^{-1}\boldsymbol{\rho}$ otherwise (both via Cholesky factoriza-

tion); the second form is used here ($P \approx 2300 > R = 1200$). The initial damping is $\mu_0 = 10^{-3}\|\mathsf{J}\|_F^2/\max(R,P)$. Damping follows Nielsen's rule [13]: with gain ratio $\varrho = (\mathcal{L} - \mathcal{L}_{\text{new}})/(\mathcal{L} - \|\boldsymbol{\rho} + \mathsf{J}\delta\boldsymbol{\theta}\|^2)$, an accepted step ($\varrho > 0$) sets $\mu \leftarrow \mu \max[1/3, 1-(2\varrho-1)^3]$ and $\nu = 2$; a rejected step sets $\mu \leftarrow \nu\mu$, $\nu \leftarrow 2\nu$ (at most 15 trials per iteration). The run stops after 600 iterations, after 30 consecutive accepted steps with relative decrease below 10^{-10}, or at the time limit.

e. Jacobian. For each point t, the function $\boldsymbol{q}(t) = (X, Y, Z, \dot{X}, \dot{Y}, \dot{Z}, f)$ is differentiated with respect to all parameters by reverse mode, vectorized over points (`torch.func.vmap` of `jacrev`); the rows of J follow by the chain rule, e.g. $\partial R_Y/\partial\boldsymbol{\theta} = \partial_{\boldsymbol{\theta}}\dot{Y} - \partial_{\boldsymbol{\theta}}X + f\,\partial_{\boldsymbol{\theta}}Z + Z\,\partial_{\boldsymbol{\theta}}f$. The cost is proportional to (number of points)$\times P$, rather than to $R\times$(cost of a full backward pass). On one CPU thread an LM iteration takes ≈ 0.5 s ($R = 1200$, $P = 2315$).

S6. CLASSICAL ESTIMATORS

a. Derivatives of the data. Noiseless data are interpolated by cubic splines. Noisy data are fitted by smoothing splines whose smoothing parameter is chosen by generalized cross-validation. Both use SciPy: the functions `CubicSpline` and `make_smoothing_spline`.

b. Direct inversion. Equation (S2) evaluated at the data times, with the denominator regularized as $Y^2 + Z^2 + 10^{-3}$.

c. Linear least squares. For a basis matrix B at the data times,

$$\boldsymbol{c} = \arg\min_{\boldsymbol{c}} \sum_{i=1}^{N} \left[\left(Z_i(\mathsf{B}\boldsymbol{c})_i - X_i + \dot{Y}_i\right)^2 + \left(Y_i(\mathsf{B}\boldsymbol{c})_i - \dot{Z}_i\right)^2 \right] + \lambda_D \|\mathsf{D}_2\boldsymbol{c}\|^2, \tag{S11}$$

solved as a linear least-squares problem. For B-splines, $\lambda_D = \{10^{-6}, 10^{-3}, 10^{-2}\} \times (n/40)^3$ for $\sigma = \{0, 0.01, 0.03\}$ (P-spline penalty [14]; the factor $(n/40)^3$ keeps the same penalty on $\int f''^2$ when the knot spacing changes). Fourier and cosine series are not penalized; truncation acts as the regularization.

d. Shooting. The field is a cubic B-spline ($n = 40$ or 80). Equation (S1) and the forward sensitivity equations $\dot{\mathsf{S}} = \mathsf{A}(t)\mathsf{S} + (0, -Z, Y)^\top \boldsymbol{b}(t)^\top$, where $\mathsf{S} = \partial\boldsymbol{r}/\partial\boldsymbol{c}$, $\mathsf{A} = \partial(\boldsymbol{\Omega}\times\boldsymbol{r})/\partial\boldsymbol{r}$ and $\boldsymbol{b}(t)$ is the basis row, are integrated with the classical fourth-order Runge–Kutta method with ten steps per data interval. The coefficients minimize $\sum_i \|\boldsymbol{r}(t_i;\boldsymbol{c}) - \boldsymbol{r}_i\|^2 + \lambda_D\|\mathsf{D}_2\boldsymbol{c}\|^2$ by Levenberg–Marquardt (SciPy `least_squares`, `method="lm"`, `xtol`=`ftol`=`gtol`= 10^{-12}, at most 200 evaluations), starting from the LSQ solution. Starting from $\boldsymbol{c} = 0$ instead can converge to a wrong local minimum (relative field error ≈ 9 for field 1 with $|0\rangle$).

S7. PROTOCOL AND REPRODUCIBILITY

a. Rules. (i) Same data for every method per (field, σ, seed); (ii) initial state $|0\rangle$; (iii) the true field never enters a loss or the choice of a hyper-parameter, and edge values are imposed only when known from the pulse design; (iv) regularization fixed a priori by σ; (v) five seeds; (vi) same state network, Adam budget and second-stage budget for every field representation; (vii) time limit of 420 s per PINN run on one CPU thread.

b. Metrics. $\epsilon_f = \|f_{\mathrm{pred}} - f\|_2/\|f\|_2$ on the N data times; $\epsilon_r = \left[\frac{1}{3N}\sum_i \|\boldsymbol{r}(t_i; f_{\mathrm{pred}}) - \boldsymbol{r}(t_i; f)\|^2\right]^{1/2}$, where $\boldsymbol{r}(\cdot; f_{\mathrm{pred}})$ is obtained by integrating Eq. (S1) with f_{pred} interpolated by a cubic spline. The RMS deviation from the noisy data is also stored.

c. What is not equalized. The number of field parameters differs between representations (Table S4); hyper-parameters were not tuned per method (in particular MixFunn, whose original training used a learning rate 10^{-2} and pruning, uses the common recipe); and the time limit penalizes representations with more expensive iterations (MixFunn reached it before converging).

d. Earlier protocol (v1). Values marked with a dagger in the main text come from an earlier version of the protocol, kept in `data/resultados/v1_*`: collocation on a fixed uniform grid of 200 points (without the data times), B-splines with $n = 40$, no edge constraint, no least-squares initialization, $\lambda_f = 10^{-10}$ without noise, and a time limit of 900 s.

e. Software and hardware. Python 3.13, PyTorch 2.15, NumPy 2.5, SciPy 1.18, Matplotlib 3.11; runs on a 4-core x86 CPU, one thread per run, four runs in parallel. The code runs unchanged on GPUs (`python run/rodar.py --device cuda`).

f. Reproduction. `python run/rodar.py --perfil padrao` (resumable; one JSON file per run in `data/resultados/padrao/jobs/`), then `python run/tabelas_artigo.py` and `python run/fig_artigo.py`.

S8. ADDITIONAL RESULTS

[**TODO:** The benchmark of the current protocol is incomplete (260 of 440 runs): missing are the PINN runs with an LM stage and noise, and the Adam-only and L-BFGS recipes for the network, Fourier, cosine and B-spline representations.]

a. Overview. Figure S3 shows the median field error of every method in every scenario of the current protocol.

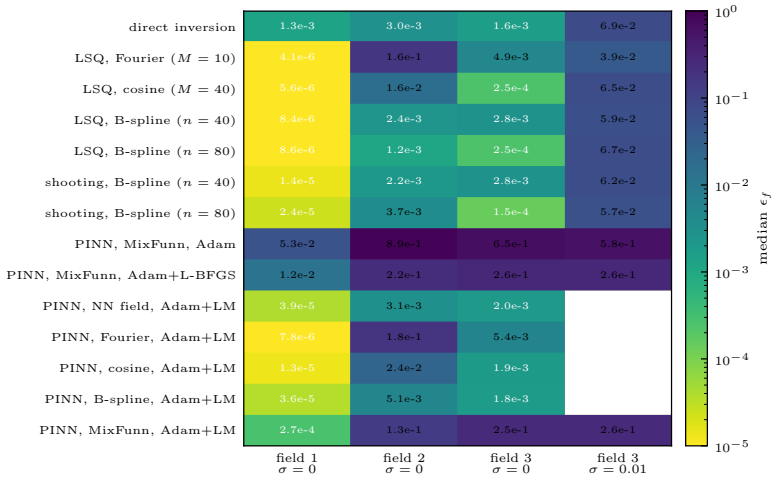


FIG. S3. Median field error ϵ_f over five seeds for every method and scenario (current protocol).

b. Optimizers. Table S6 lists the field error for field 3 after each second-stage optimizer (earlier protocol; Fig. 5 of the main text).

TABLE S6. Field 3, no noise: relative field error after 3000 Adam steps and each second stage (median and range over five seeds; earlier protocol, see Sec. S7).

second stage	NN field		B-spline field	
	ϵ_f median	[min, max]	ϵ_f median	[min, max]
none (Adam only)	0.26	[0.25, 0.26]	0.25	[0.25, 0.27]
L-BFGS	3.3×10^{-2}	$[3.1 \times 10^{-2}, 4.4 \times 10^{-2}]$	2.5×10^{-2}	$[1.4 \times 10^{-2}, 5.9 \times 10^{-2}]$
self-scaled BFGS	0.10	$[7.4 \times 10^{-2}, 0.15]$	9.9×10^{-2}	$[4.7 \times 10^{-2}, 0.16]$
dense BFGS	1.3×10^{-2}	$[6.7 \times 10^{-3}, 1.7 \times 10^{-2}]$	1.1×10^{-2}	$[8.7 \times 10^{-3}, 2.3 \times 10^{-2}]$
LM, causal weights in Adam	1.7×10^{-3}	$[8.9 \times 10^{-4}, 2.0 \times 10^{-3}]$	7.2×10^{-3}	$[5.2 \times 10^{-3}, 8.4 \times 10^{-3}]$
LM	1.0×10^{-3}	$[9.2 \times 10^{-4}, 2.1 \times 10^{-3}]$	7.6×10^{-3}	$[5.4 \times 10^{-3}, 9.3 \times 10^{-3}]$

c. Per-scenario results. Tables S7–S10 and Figs. S4–S7 give both metrics, their range over seeds and the mean run time for every method.

TABLE S7. Field 1, no noise.

method	ϵ_f median	ϵ_f [min, max]	ϵ_r median	time (s)
LSQ, Fourier (M=10)	4.1×10^{-6}	$[4.1 \times 10^{-6}, 4.1 \times 10^{-6}]$	5.5×10^{-7}	0
LSQ, cosine (M=40)	5.6×10^{-6}	$[5.6 \times 10^{-6}, 5.6 \times 10^{-6}]$	5.1×10^{-7}	0
Fourier (M=10), Adam + LM	7.8×10^{-6}	$[2.0 \times 10^{-6}, 1.6 \times 10^{-5}]$	5.0×10^{-7}	311
LSQ, B-spline (n=40)	8.4×10^{-6}	$[8.4 \times 10^{-6}, 8.4 \times 10^{-6}]$	6.0×10^{-7}	0
LSQ, B-spline (n=80)	8.6×10^{-6}	$[8.6 \times 10^{-6}, 8.6 \times 10^{-6}]$	6.9×10^{-7}	0
cosine (M=40), Adam + LM	1.3×10^{-5}	$[8.5 \times 10^{-6}, 2.3 \times 10^{-5}]$	6.0×10^{-7}	314
shooting, B-spline (n=40)	1.4×10^{-5}	$[1.4 \times 10^{-5}, 1.4 \times 10^{-5}]$	1.4×10^{-7}	0
shooting, B-spline (n=80)	2.4×10^{-5}	$[2.4 \times 10^{-5}, 2.4 \times 10^{-5}]$	9.4×10^{-7}	0
B-spline (n=80), Adam + LM	3.6×10^{-5}	$[9.7 \times 10^{-6}, 5.9 \times 10^{-5}]$	6.7×10^{-7}	318
NN field, Adam + LM	3.9×10^{-5}	$[2.3 \times 10^{-5}, 4.8 \times 10^{-5}]$	8.9×10^{-7}	288
MixFunn, Adam + LM	2.7×10^{-4}	$[1.7 \times 10^{-4}, 7.9 \times 10^{-4}]$	1.5×10^{-5}	422
direct inversion	1.3×10^{-3}	$[1.3 \times 10^{-3}, 1.3 \times 10^{-3}]$	1.2×10^{-3}	0
MixFunn, Adam + L-BFGS	1.2×10^{-2}	$[1.1 \times 10^{-2}, 2.7 \times 10^{-2}]$	6.9×10^{-4}	55
MixFunn, Adam only	5.3×10^{-2}	$[3.2 \times 10^{-2}, 0.35]$	8.8×10^{-3}	80

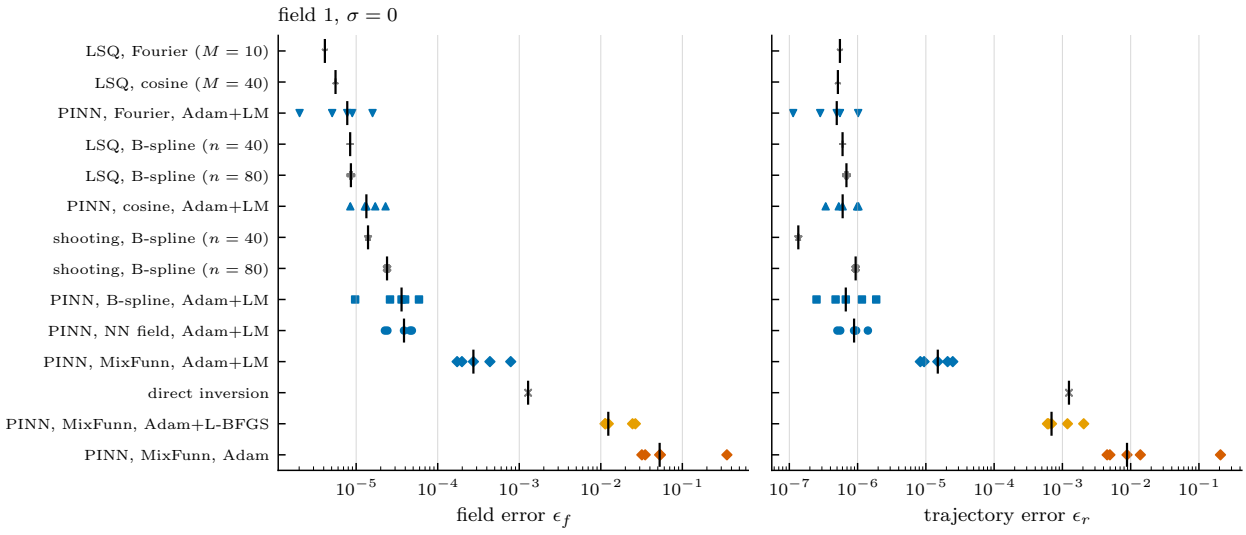


FIG. S4. Field 1, no noise: field error and trajectory error of every run (points) and their medians (bars). Colour: optimizer; marker: field representation.

TABLE S8. Field 2, no noise.

method	ϵ_f median	ϵ_f [min, max]	ϵ_r median	time (s)
LSQ, B-spline (n=80)	1.2×10^{-3}	[1.2×10^{-3}, 1.2×10^{-3}]	6.2×10^{-5}	0
shooting, B-spline (n=40)	2.2×10^{-3}	[2.2×10^{-3}, 2.2×10^{-3}]	5.9×10^{-5}	0
LSQ, B-spline (n=40)	2.4×10^{-3}	[2.4×10^{-3}, 2.4×10^{-3}]	1.6×10^{-4}	0
direct inversion	3.0×10^{-3}	[3.0×10^{-3}, 3.0×10^{-3}]	1.7×10^{-3}	0
NN field, Adam + LM	3.1×10^{-3}	[1.6×10^{-3}, 5.1×10^{-3}]	1.1×10^{-4}	290
shooting, B-spline (n=80)	3.7×10^{-3}	[3.7×10^{-3}, 3.7×10^{-3}]	1.5×10^{-4}	0
B-spline (n=80), Adam + LM	5.1×10^{-3}	[4.5×10^{-3}, 1.5×10^{-2}]	2.2×10^{-4}	319
LSQ, cosine (M=40)	1.6×10^{-2}	[1.6×10^{-2}, 1.6×10^{-2}]	4.4×10^{-3}	0
cosine (M=40), Adam + LM	2.4×10^{-2}	[2.2×10^{-2}, 2.8×10^{-2}]	6.7×10^{-4}	313
MixFunn, Adam + LM	0.13	[4.8×10^{-2}, 0.18]	1.0×10^{-2}	422
LSQ, Fourier (M=10)	0.16	[0.16, 0.16]	5.5×10^{-2}	0
Fourier (M=10), Adam + LM	0.18	[0.18, 0.18]	1.0×10^{-2}	309
MixFunn, Adam + L-BFGS	0.22	[0.21, 0.33]	1.8×10^{-2}	51
MixFunn, Adam only	0.89	[0.32, 72]	0.29	77

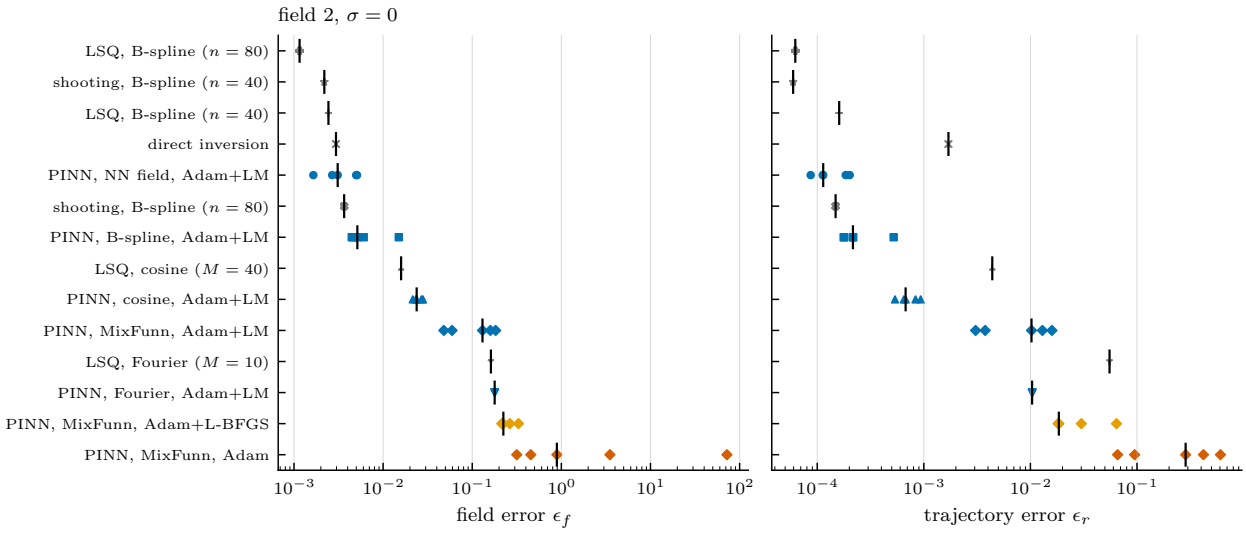


FIG. S5. Field 2, no noise; as Fig. S4.

TABLE S9. Field 3, no noise.

method	ϵ_f median	ϵ_f [min, max]	ϵ_r median	time (s)
shooting, B-spline (n=80)	1.5×10^{-4}	$[1.5 \times 10^{-4}, 1.5 \times 10^{-4}]$	4.1×10^{-6}	0
LSQ, cosine (M=40)	2.5×10^{-4}	$[2.5 \times 10^{-4}, 2.5 \times 10^{-4}]$	2.6×10^{-5}	0
LSQ, B-spline (n=80)	2.5×10^{-4}	$[2.5 \times 10^{-4}, 2.5 \times 10^{-4}]$	1.7×10^{-5}	0
direct inversion	1.6×10^{-3}	$[1.6 \times 10^{-3}, 1.6 \times 10^{-3}]$	4.7×10^{-4}	0
B-spline (n=80), Adam + LM	1.8×10^{-3}	$[1.0 \times 10^{-3}, 2.0 \times 10^{-3}]$	4.2×10^{-5}	313
cosine (M=40), Adam + LM	1.9×10^{-3}	$[1.1 \times 10^{-3}, 2.2 \times 10^{-3}]$	4.9×10^{-5}	307
NN field, Adam + LM	2.0×10^{-3}	$[9.3 \times 10^{-4}, 2.2 \times 10^{-3}]$	5.8×10^{-5}	287
shooting, B-spline (n=40)	2.8×10^{-3}	$[2.8 \times 10^{-3}, 2.8 \times 10^{-3}]$	4.2×10^{-5}	0
LSQ, B-spline (n=40)	2.8×10^{-3}	$[2.8 \times 10^{-3}, 2.8 \times 10^{-3}]$	4.6×10^{-5}	0
LSQ, Fourier (M=10)	4.9×10^{-3}	$[4.9 \times 10^{-3}, 4.9 \times 10^{-3}]$	4.6×10^{-4}	0
Fourier (M=10), Adam + LM	5.4×10^{-3}	$[5.4 \times 10^{-3}, 5.5 \times 10^{-3}]$	2.6×10^{-4}	299
MixFunn, Adam + LM	0.25	[0.17, 0.25]	1.2×10^{-2}	422
MixFunn, Adam + L-BFGS	0.26	[0.25, 0.26]	1.3×10^{-2}	73
MixFunn, Adam only	0.65	[0.32, 55]	0.16	82

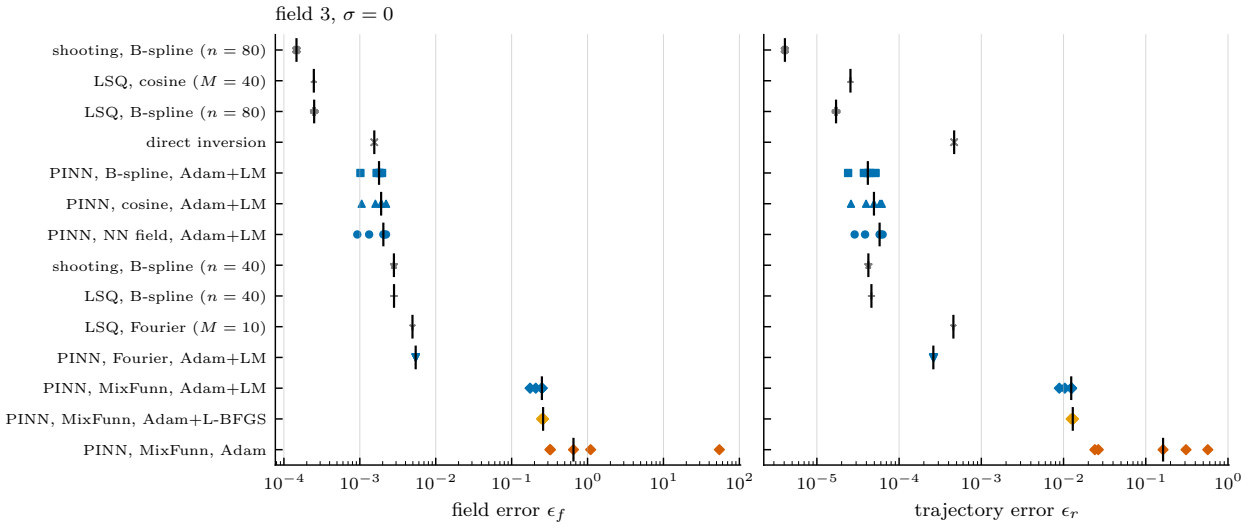


FIG. S6. Field 3, no noise; as Fig. S4.

TABLE S10. Field 3, noise $\sigma = 0.01$.

method	ϵ_f median	ϵ_f [min, max]	ϵ_r median	time (s)
LSQ, Fourier (M=10)	3.9×10^{-2}	$[2.9 \times 10^{-2}, 5.3 \times 10^{-2}]$	4.6×10^{-3}	0
shooting, B-spline (n=80)	5.7×10^{-2}	$[4.8 \times 10^{-2}, 9.6 \times 10^{-2}]$	2.9×10^{-3}	1
LSQ, B-spline (n=40)	5.9×10^{-2}	$[5.6 \times 10^{-2}, 6.7 \times 10^{-2}]$	5.0×10^{-3}	0
shooting, B-spline (n=40)	6.2×10^{-2}	$[5.4 \times 10^{-2}, 0.11]$	3.1×10^{-3}	0
LSQ, cosine (M=40)	6.5×10^{-2}	$[6.1 \times 10^{-2}, 7.2 \times 10^{-2}]$	4.9×10^{-3}	0
LSQ, B-spline (n=80)	6.7×10^{-2}	$[6.5 \times 10^{-2}, 7.2 \times 10^{-2}]$	5.0×10^{-3}	0
direct inversion	6.9×10^{-2}	$[6.8 \times 10^{-2}, 7.5 \times 10^{-2}]$	4.9×10^{-3}	0
MixFunn, Adam + LM	0.26	$[4.8 \times 10^{-2}, 0.26]$	1.3×10^{-2}	423
MixFunn, Adam + L-BFGS	0.26	[0.24, 0.27]	1.3×10^{-2}	119
MixFunn, Adam only	0.58	[0.31, 1.2]	0.13	140

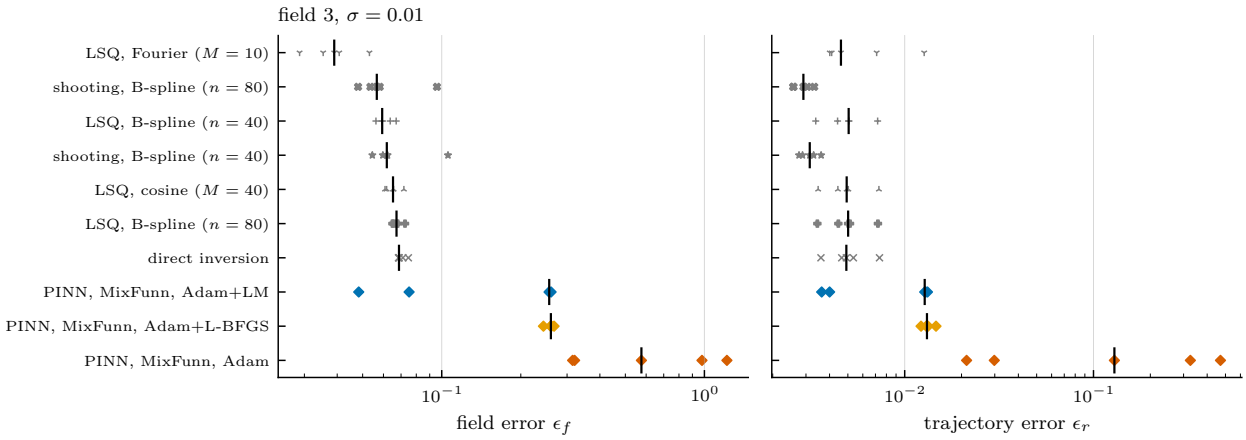


FIG. S7. Field 3, noise $\sigma = 0.01$; as Fig. S4.

S9. OUTLOOK SETTINGS: DETAILS

a. Two-qubit model. The two-qubit notebooks of the repository (`notebooks/modelo_2(Rede neural)/PINN_field_4_twoqubits_*.ipynb`) use

$$H(t) = \omega_2\,\sigma_z \otimes \mathbb{1} + \omega_1\,\mathbb{1} \otimes \sigma_z + f(t)\,(\sigma_x \otimes \sigma_x + \sigma_y \otimes \sigma_y), \tag{S12}$$

with $\omega_1 = \omega_2 = 0.5$, field 3 with $T = 2\pi$, $N = 200$ time points, initial state $|+\rangle|+\rangle$, the fifteen observables $\langle \sigma_\mu \otimes \sigma_\nu \rangle$ ($(\mu,\nu) \neq (0,0)$), and, in the open case, Lindblad operators $\sqrt{\gamma}\,\sigma_\pm$ on each qubit with $\gamma = 0.05$ [15, 16]. The expectation values obey $\frac{d}{dt}\langle S\rangle = i\langle [H,S]\rangle + \sum_k \gamma_k \langle \mathcal{L}_k^\dagger S \mathcal{L}_k - \frac{1}{2}\{S, \mathcal{L}_k^\dagger \mathcal{L}_k\}\rangle$, the form used in the PINNverse framework [17, 18].

b. Blind states and phase dependence. We checked numerically (QuTiP, tolerance 10^{-12}) that (i) starting from $|00\rangle$, the fifteen expectation values for f and $f/2$ coincide exactly, both for $\omega_1 = \omega_2 = 0.5$ and for $\omega_1 = 0.5$, $\omega_2 = 0.7$, whereas from $|01\rangle$ and $|{+}{+}\rangle$ they differ by up to 1.7 and 0.97; and (ii) for $\omega_1 = \omega_2$, where $\|[H_0, H_1]\| = 0$, the expectation values equal those of $e^{-iH_0 t} e^{-i\Phi(t) H_1} |{+}{+}\rangle$ with $\Phi(t) = \int_0^t f$ to 1.7×10^{-7}. The data therefore determine $\Phi(t)$, and $f = \dot{\Phi}$.

c. Partial information. With only $\langle \sigma_z \rangle$ measured, the PINN loss keeps the full Bloch equations for (X, Y, Z) and restricts the data term to Z; shooting restricts its residual to Z; direct inversion and LSQ are not applicable. The symmetries of Sec. S2 require an initial state with X, Y, Z all nonzero, or two measured components. The planned extension of the protocol is described in `notes/TODO.md` of the repository.

---

[1] J. R. Johansson, P. D. Nation, and F. Nori, Comput. Phys. Commun. **184**, 1234 (2013).

[2] C. de Boor, *A Practical Guide to Splines*, revised ed. (Springer, New York, 2001).

[3] T. de Souza Farias, G. G. de Lima, J. Maziero, and C. J. Villas-Boas, MixFunn: A neural network for differential equations with improved generalization and interpretability (2025), arXiv:2503.22528.

[4] G. H. Golub and V. Pereyra, SIAM J. Numer. Anal. **10**, 413 (1973).

[5] S. Wang, S. Sankaran, and P. Perdikaris, Respecting causality is all you need for training physics-informed neural networks (2022), arXiv:2203.07404.

[6] S. Wang, Y. Teng, and P. Perdikaris, SIAM J. Sci. Comput. **43**, A3055 (2021).

[7] D. P. Kingma and J. Ba, Adam: A method for stochastic optimization (2015), iCLR 2015, arXiv:1412.6980.

[8] D. C. Liu and J. Nocedal, Math. Program. **45**, 503 (1989).

[9] J. F. Urbán, P. Stefanou, and J. A. Pons, J. Comput. Phys. (2025), arXiv:2405.04230.

[10] E. Kiyani, K. Shukla, J. F. Urbán, J. Darbon, and G. E. Karniadakis, Which optimizer works best for physics-informed neural networks and Kolmogorov-Arnold networks? (2025), arXiv:2501.16371.

[11] K. Levenberg, Q. Appl. Math. **2**, 164 (1944).

[12] D. W. Marquardt, J. Soc. Ind. Appl. Math. **11**, 431 (1963).

[13] H. B. Nielsen, *Damping Parameter in Marquardt's Method*, Tech. Rep. IMM-REP-1999-05 (Informatics and Mathematical Modelling, Technical University of Denmark, 1999).

[14] P. H. C. Eilers and B. D. Marx, Stat. Sci. **11**, 89 (1996).

[15] G. Lindblad, Commun. Math. Phys. **48**, 119 (1976).

[16] V. Gorini, A. Kossakowski, and E. C. G. Sudarshan, J. Math. Phys. **17**, 821 (1976).

[17] L. K. Castelano, I. Cunha, F. S. Luiz, R. de Jesus Napolitano, M. V. S. Prado, and F. F. Fanchini, Phys. Rev. A **110**, 032607 (2024).

[18] G. G. de Lima, I. Cunha, and L. K. Castelano, Inverse physics-informed neural networks procedure for detecting noise in open quantum systems (2025), arXiv:2507.12552 [quant-ph].